NAModule1Assignment

October 15, 2026

1 Module 1 Assignment

In this module, you learned about topics such as homophily, structural holes, and K-core decomposition. In this assignment, you will use NetworkX's library of functions relating to these topics.

[5]:
```
import networkx as nx
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import adjusted_rand_score
```

2 Part 1: Homophily, Assortative mixing

At football headquarters, we have a network representing the American football season of the year 2001, where nodes are teams and we form an undirected edge between teams that have played against each other at least once during the season. We also have additional about each team as node attributes, including the team ID, team name, number of wins, number of losses, and the conference number. Below is an example of a node in the dataset:

```
node [
    id 1
    label "FloridaState"
    wins 11
    losses 2
    conference "Atlantic Coast"
]
```

This node represents the Florida State team wth ID 1, 11 wins, 2 losses, and conference ID 0.

In American Football, conferences are groups of teams that are in a league together and play against each other to win the league title. It is not uncommon however, for teams to play a few teams that are outside of their conference.

3 Task 1a:

Somehow, a disgruntled fan has hacked into our system and created a second corrupted graph with randomized edges. We do not know which is which, and we need your help to pick which one is the correct network.

Given that teams in conferences together tend to play each other, we know we can use one of the assortativity functions we learned about this week to deduce which graph is the uncorrupted one. It is your job to figure out which assortativity function and which feature of the nodes are relevant, return the proper coefficient, and decide which is the uncorrupted graph.

The function below should return a tuple containing the coefficient for football1.gml and football2.gml in that order.

[6]:
```
G_football1 = nx.read_gml('assets/football1.gml', label='id')

G_football2 = nx.read_gml('assets/football2.gml', label='id')
```

[7]:
```
task_id = '1'

## Utility Code will go here
```

[8]:
```
def task_1a_solution():
    g1_coeff, g2_coeff = 0, 0

    # YOUR CODE HERE
    # raise NotImplementedError()
    g1_coeff, g2_coeff = nx.attribute_assortativity_coefficient(G_football1,␣
 ↪"conference"), nx.attribute_assortativity_coefficient(G_football2,␣
 ↪"conference")



    return g1_coeff, g2_coeff
```

[9]:
```
# Use this cell to explore your solution.

task_1a_solution()
```

[9]:
```
(0.6275381679111908, 0.009602406049995276)
```

[10]:
```
print(f"Task {task_id} - AG tests")

stu_ans = task_1a_solution()

print(f"Task {task_id} - your answer:\n{stu_ans}")

assert isinstance(
    stu_ans, tuple
), f"Task {task_id}: Your function should return a tuple."

assert len(stu_ans) == 2, f"Task {task_id}: Your returned tuple should contain␣
 ↪exactly two elements."

assert all(
```

```
    isinstance(num, float) for num in stu_ans
), f"Task {task_id}: Both elements in the returned tuple should be floats."

assert all(
    -1.0 <= num <= 1.0 for num in stu_ans
), f"Task {task_id}: Assortativity coefficients should be between -1 and 1."
```

```
Task 1 - AG tests
Task 1 - your answer:
(0.6275381679111908, 0.009602406049995276)
```

4 Deciding Real_G_football

Now, given the assortativity coefficient, you should be able to deduce which network is the real one. Assign the real one to Real_G_football below.

[11]:
```
Real_G_football = G_football1
# YOUR CODE HERE
#raise NotImplementedError()
```

5 Task 1b:

We want to see if football teams in our network tend to play other teams who play a similar number of teams. That is, teams that play many (few) teams play against teams that also play many (few) teams. Think about which assortativity coefficient would help us accomplish this, and return it in the function below as a float.

[12]:
```
task_id = '1b'

## Utility Code will go here
```

[13]:
```
def task_1b_solution():
    sol = nx.degree_assortativity_coefficient(Real_G_football)

    # YOUR CODE HERE
    #raise NotImplementedError()

    return sol
```

[14]:
```
# Use this cell to explore your solution.

task_1b_solution()
```

[14]:
```
0.16244224957444287
```

[15]:
```
print(f"Task {task_id} - AG tests")

stu_ans = task_1b_solution()

print(f"Task {task_id} - your answer:\n{stu_ans}")

assert isinstance(
    stu_ans, float
), f"Task {task_id}: Your function should return a float."

assert (
    -1.0 <= stu_ans <= 1.0
), f"Task {task_id}: The degree assortativity coefficient should be between -1␣
 ↪and 1."
```

```
Task 1b - AG tests
Task 1b - your answer:
0.16244224957444287
```

6 Task 1c:

As a final task for this task, we want to know if teams tend to play against teams with similar win counts. Think about which coefficient score would help us measure this, and then return that score in the function below as a float.

[16]:
```
task_id = '1c'

## Utility Code will go here
```

[17]:
```
def task_1c_solution():
    sol = nx.numeric_assortativity_coefficient(Real_G_football, 'wins')

    # YOUR CODE HERE
    #raise NotImplementedError()

    return sol
```

[18]:
```
# Use this cell to explore your solution.

task_1c_solution()
```

[18]:
```
-0.049806582644503085
```

[19]:
```
print(f"Task {task_id} - AG tests")

stu_ans = task_1c_solution()
```

```
print(f"Task {task_id} - your answer:\n{stu_ans}")

assert isinstance(
    stu_ans, float
), f"Task {task_id}: Your function should return a float."

assert (
    -1.0 <= stu_ans <= 1.0
), f"Task {task_id}: The numeric assortativity coefficient should be between -1␣
 ↪and 1."
```

```
Task 1c - AG tests
Task 1c - your answer:
-0.049806582644503085
```

7 Part 2: Structural holes

8 Task 2a:

In American football, teams usually play within their conference. However, some teams occasionally play against teams from other conferences. These teams can be viewed as brokers spanning structural holes.

Your task here is to find the brokers in our network and return a dictionary containing only the brokers, where the key is a string representing the name of the college, and the value is a float representing the constraint coefficient.

For this task, we will consider a team to be a broker if they have a constraint coefficient of less than 0.15.

[20]:
```
task_id = '2a'

## Utility Code will go here
```

[21]:
```
def task_2a_solution():
    threshold = 0.15
    constraints = nx.constraint(Real_G_football)
    brokers = {G_football1.nodes[key]['label']:val for (key, val) in␣
 ↪constraints.items() if val < 0.15}
    # YOUR CODE HERE
    #raise NotImplementedError()

    return brokers
```

[22]:
```
# Use this cell to explore your solution.

task_2a_solution()
```

[22]:
```
{'PennState': 0.14832231723803696,
 'LouisianaTech': 0.1257241289154168,
 'MiddleTennesseeState': 0.14153753443526168,
 'NewMexicoState': 0.14415067487631977,
 'Navy': 0.12577556177856705,
 'NotreDame': 0.11802929748271598}
```

[23]:
```
print(f"Task {task_id} - AG tests")

stu_ans = task_2a_solution()

print(f"Task {task_id} - your answer:\n{stu_ans}")

assert isinstance(
    stu_ans, dict
), f"Task {task_id}: Your function should return a dictionary."

assert all(
    isinstance(k, str) for k in stu_ans.keys()
), f"Task {task_id}: The dictionary keys should be strings representing node␣
 ↪labels."

assert all(
    isinstance(v, float) for v in stu_ans.values()
), f"Task {task_id}: The dictionary values should be floats representing␣
 ↪constraint scores."

assert all(
    v < 0.15 for v in stu_ans.values()
), f"Task {task_id}: All constraint values should be below 0.15."
```

```
Task 2a - AG tests
Task 2a - your answer:
{'PennState': 0.14832231723803696, 'LouisianaTech': 0.1257241289154168,
'MiddleTennesseeState': 0.14153753443526168, 'NewMexicoState':
0.14415067487631977, 'Navy': 0.12577556177856705, 'NotreDame':
0.11802929748271598}
```

9 Task 2b

Some fans are criticizing teams who don't play out of their conference, and say that they don't play against other teams so they win more. To ascertain the validity of this statement, we would like you to compute the following:

- The average win rates of the bottom 10 brokers (`bottom_10_broker_avg_wr` in the function below).
- The average win rates of the top 10 brokers (`top_10_broker_avg_wr` in the function below).

- The correlation coefficient between the win rates of all teams and their constraint coefficients (`correlation_coefficient` in the function below).

Complete the function below so the appropriate values are returned. The return format has already been defined for you below.

Hints:

- `sorted_constraints_list` contains a list of tuples, where the first value is a node id, and the second is the constraint coefficient associated with that node id.
- Loop through `sorted_constraints_list`'s nodes to create a list containing the win rate of each node. Since `sorted_constraints_list` is sorted, you'll see the nodes in order from smallest constraint coefficient to largest.
- Now that you have a list containing the win rates of each node, you can take the first 10 and last 10 items in the list you made to calculate the average win rates of the top and bottom 10 brokers.
- You can also use the win rate list and a list containing `sorted_constraints_list`'s values to calculate the correlation coefficient.
- A lower constraint coefficient means the team is more of a broker.
- For this question, win rate will equal the number of wins divided by the number of wins plus losses.

[24]:
```
task_id = '2b'

## Utility Code will go here
```

[25]:
```
def task_2b_solution():
    constraints = nx.constraint(Real_G_football)
    sorted_constraints_list = (sorted(constraints.items(), key=lambda item:␣
 ↪item[1]))

    top_10_broker_avg_wr, bottom_10_broker_avg_wr, correlation_coefficient = 0,␣
 ↪0, 0

    # YOUR CODE HERE
    #raise NotImplementedError()
    win_rates = []
    for (id, cons) in sorted_constraints_list:
        win_rates.append(G_football1.nodes[id]['wins'] / (G_football1.
 ↪nodes[id]['wins'] + G_football1.nodes[id]['losses']))
    top_10_broker_avg_wr, bottom_10_broker_avg_wr = np.mean(win_rates[:10]), np.
 ↪mean(win_rates[-10:])
    constraints = [c for (id, c) in sorted_constraints_list]
    correlation_coefficient = np.corrcoef(constraints, win_rates)[0,1]
    #print(constraints)
    #print(win_rates)
```

```
    return top_10_broker_avg_wr, bottom_10_broker_avg_wr,␣
 ↪correlation_coefficient
```

[26]:
```
# Use this cell to explore your solution.

task_2b_solution()
```

[26]:
```
(0.4531052281052281, 0.5325757575757575, 0.0901664589975476)
```

[27]:
```
print(f"Task {task_id} - AG tests")

stu_ans = task_2b_solution()

print(f"Task {task_id} - your answer:\n{stu_ans}")

assert isinstance(stu_ans, tuple), f"Task {task_id}: Your function should␣
 ↪return a tuple."
assert len(stu_ans) == 3, f"Task {task_id}: Your returned tuple should have␣
 ↪three elements."

top_10_broker_avg_wr, bottom_10_broker_avg_wr, correlation_coefficient = stu_ans

assert isinstance(
    top_10_broker_avg_wr, float
), f"Task {task_id}: The first element of the returned tuple should be a float."
assert isinstance(
    bottom_10_broker_avg_wr, float
), f"Task {task_id}: The second element of the returned tuple should be a float.
 ↪"

assert isinstance(
    correlation_coefficient, float
), f"Task {task_id}: The third element of the returned tuple should be a float."
assert -1 <= correlation_coefficient <= 1, f"Task {task_id}: The correlation␣
 ↪coefficient should be between -1 and 1."
```

```
Task 2b - AG tests
Task 2b - your answer:
(0.4531052281052281, 0.5325757575757575, 0.0901664589975476)
```

10 Task 2b Supplemental Question:

Do the top brokers tend to win more? Return the answer as a true or false boolean value in the function below.

[50]:
```
task_id = '2bs'

def task_2b_supplemental_solution():
    sol = False

    # YOUR CODE HERE
    #raise NotImplementedError()

    return sol
```

[51]:
```
print(f"Task {task_id} - AG tests")

stu_ans = task_2b_supplemental_solution()

print(f"Task {task_id} - your answer:\n{stu_ans}")

assert isinstance(
    stu_ans, bool
), f"Task {task_id}: Your function should return a bool."
```

```
Task 2bs - AG tests
Task 2bs - your answer:
False
```

11 Task 2c:

Due to budget cuts, the NCAA must remove three teams from the league. To minimize the impact on the diversity of matchups, the organization aims to cut the teams that contribute the least unique connectivity within the game network.

In network terms, a team's effective size reflects the extent to which its opponents provide unique rather than redundant connections. A team with a low effective size primarily competes against teams that already play against each other frequently, meaning its removal would have minimal impact on the variety of matchups in the league.

To determine which teams should be removed, identify the three teams with the lowest effective size scores. Return the names of these teams as a list of strings, ordered from the lowest to highest effective size value.

[52]:
```
task_id = '2c'

## Utility Code will go here
```

[28]:
```
def task_2c_solution():
    sol = None

    # YOUR CODE HERE
    #raise NotImplementedError()
```

```
    eff_size = nx.effective_size(Real_G_football)
    sorted_esize_list = (sorted(eff_size.items(), key=lambda item: item[1]))
    #print(sorted_esize_list)
    sol = [G_football1.nodes[id]['label'] for (id, s) in sorted_esize_list[:3]]

    return sol
```

[29]:
```
task_2c_solution()
```

[29]:
```
['WakeForest', 'Virginia', 'Clemson']
```

[30]:
```
print(f"Task {task_id} - AG tests")

stu_ans = task_2c_solution()

print(f"Task {task_id} - your answer:\n{stu_ans}")

assert isinstance(
    stu_ans, list
), f"Task {task_id}: Your function should return a list."

assert len(stu_ans) == 3, f"Task {task_id}: Your list should contain exactly 3␣
 ↪elements."

assert all(
    isinstance(team, str) for team in stu_ans
), f"Task {task_id}: All elements in the list should be strings (team labels)."
```

```
Task 2b - AG tests
Task 2b - your answer:
['WakeForest', 'Virginia', 'Clemson']
```

12 Part 3: Dolphins

13 Task 3a:

Dolphins are social creatures. We have a network of friendships stored in the file dolphins.gml. However, due to experimentation by the evil Dr. Romero, the dolphins become very judgemental and start kicking dolphins who don't have a certain number of friends from their pods. On day 0, the dolphins kick out every dolphin that doesn't have at least 0 friends, so every dolphin survives. On day 1, the dolphins kick out every dolphin that doesn't have at least 1 friend. On day two, they kick out every dolphin who doesn't have at least two friends. This goes on until all dolphins will be kicked out if they increase the required number of friends to stay in the pod.

Note: If a dolphin doesn't have enough friends after its friends are kicked out on a day, that dolphin is also kicked out on that same day. For example, let's say on day two, dolphin A is friends with only dolphins B and C, and dolphins B and C are friends with only dolphin A. When the dolphins

are kicked out, dolphins B and C will be kicked out because they only have one friend, dolphin A. But then, dolphin A will have no friends left, so dolphin A will be kicked from the group as well.

You need to make and return a list of lists, where the first list contains the names of the dolphins that survive the first day, and the second list contains the names of those who survive the second day, and so on. You should only keep adding lists while there are still survivors (you should not have an empty list at the end of the list)

You should also return a list containing the percentage of dolphins that have survived each day while there are still survivors (you should not end the list with 0.0). When calculating the survival percentage list, use the total number of dolphins in the dataset as the source of comparison.

Ex. If we started with 10 dolphins and 7 dolphins survived the first day and 5 were left on the second, our list would be [1.0, 0.7, 0.5].

Hint: Which algorithm from this module's lectures would help you accomplish this easily?

[31]:
```
G_dolphins = nx.read_gml("assets/dolphins.gml")
```

[32]:
```
task_id = '3a'

## Utility Code will go here
```

[39]:
```
def task_3a_solution():
    max_core_number = max(nx.core_number(G_dolphins).values())
    daily_survival_list = [[] for _ in range(max_core_number)]
    for k in range(max_core_number):
        daily_survival_list[k] = list(nx.k_core(G_dolphins, k+1).nodes())
    daily_survival_percentage = [len(daily_survival_list[k]) / len(G_dolphins.
 ↪nodes()) for k in range(len(daily_survival_list))]

    # YOUR CODE HERE
    #raise NotImplementedError()

    return daily_survival_list, daily_survival_percentage
```

[40]:
```
task_3a_solution()
```

[40]:
```
([['Beak',
   'Beescratch',
   'Bumper',
   'CCL',
   'Cross',
   'DN16',
   'DN21',
   'DN63',
   'Double',
   'Feather',
   'Fish',
```

```
'Five',
'Fork',
'Gallatin',
'Grin',
'Haecksel',
'Hook',
'Jet',
'Jonah',
'Knit',
'Kringel',
'MN105',
'MN23',
'MN60',
'MN83',
'Mus',
'Notch',
'Number1',
'Oscar',
'Patchback',
'PL',
'Quasi',
'Ripplefluke',
'Scabs',
'Shmuddel',
'SMN5',
'SN100',
'SN4',
'SN63',
'SN89',
'SN9',
'SN90',
'SN96',
'Stripes',
'Thumper',
'Topless',
'TR120',
'TR77',
'TR82',
'TR88',
'TR99',
'Trigger',
'TSN103',
'TSN83',
'Upbang',
'Vau',
'Wave',
'Web',
```

```
 'Whitetip',
 'Zap',
 'Zig',
 'Zipfel'],
['Beak',
 'Beescratch',
 'Bumper',
 'CCL',
 'DN16',
 'DN21',
 'DN63',
 'Double',
 'Feather',
 'Fish',
 'Gallatin',
 'Grin',
 'Haecksel',
 'Hook',
 'Jet',
 'Jonah',
 'Knit',
 'Kringel',
 'MN105',
 'MN60',
 'MN83',
 'Mus',
 'Notch',
 'Number1',
 'Oscar',
 'Patchback',
 'PL',
 'Ripplefluke',
 'Scabs',
 'Shmuddel',
 'SN100',
 'SN4',
 'SN63',
 'SN89',
 'SN9',
 'SN90',
 'SN96',
 'Stripes',
 'Thumper',
 'Topless',
 'TR120',
 'TR77',
 'TR88',
```

```
 'TR99',
 'Trigger',
 'TSN103',
 'TSN83',
 'Upbang',
 'Vau',
 'Wave',
 'Web',
 'Zap',
 'Zipfel'],
['Beak',
 'Beescratch',
 'Bumper',
 'CCL',
 'DN16',
 'DN21',
 'DN63',
 'Double',
 'Feather',
 'Fish',
 'Gallatin',
 'Grin',
 'Haecksel',
 'Hook',
 'Jet',
 'Jonah',
 'Knit',
 'Kringel',
 'MN105',
 'MN60',
 'MN83',
 'Mus',
 'Notch',
 'Number1',
 'Oscar',
 'Patchback',
 'PL',
 'Scabs',
 'Shmuddel',
 'SN100',
 'SN4',
 'SN63',
 'SN9',
 'SN90',
 'SN96',
 'Stripes',
 'Thumper',
```

```
  'Topless',
  'TR77',
  'TR99',
  'Trigger',
  'TSN103',
  'Upbang',
  'Web',
  'Zap'],
 ['Beak',
  'Beescratch',
  'DN21',
  'DN63',
  'Double',
  'Feather',
  'Fish',
  'Gallatin',
  'Grin',
  'Haecksel',
  'Hook',
  'Jet',
  'Jonah',
  'Knit',
  'Kringel',
  'MN105',
  'MN83',
  'Oscar',
  'Patchback',
  'PL',
  'Scabs',
  'SN100',
  'SN4',
  'SN63',
  'SN9',
  'SN90',
  'SN96',
  'Stripes',
  'Topless',
  'TR77',
  'TR99',
  'Trigger',
  'TSN103',
  'Upbang',
  'Web',
  'Zap']],
[1.0, 0.8548387096774194, 0.7258064516129032, 0.5806451612903226])
```

[41]:
```
print(f"Task {task_id} - AG tests")

stu_ans = task_3a_solution()

print(f"Task {task_id} - your answer:\n{stu_ans}")

assert isinstance(stu_ans, tuple), f"Task {task_id}: Your function should␣
 ↪return a tuple."
assert len(stu_ans) == 2, f"Task {task_id}: Your returned tuple should have two␣
 ↪elements."

daily_survival_list, daily_survival_percentage = stu_ans

assert isinstance(
    daily_survival_list, list
), f"Task {task_id}: The first element of the returned tuple should be a list."
assert all(
    isinstance(day_list, list) for day_list in daily_survival_list
), f"Task {task_id}: Each element in `daily_survival_list` should be a list."

assert isinstance(
    daily_survival_percentage, list
), f"Task {task_id}: The second element of the returned tuple should be a list."
assert all(
    isinstance(p, float) for p in daily_survival_percentage
), f"Task {task_id}: Each element in `daily_survival_percentage` should be a␣
 ↪float."

assert all(
    0 <= p <= 1 for p in daily_survival_percentage
), f"Task {task_id}: Values in `daily_survival_percentage` should be between 0␣
 ↪and 1."
```

```
Task 3a - AG tests
Task 3a - your answer:
([['Beak', 'Beescratch', 'Bumper', 'CCL', 'Cross', 'DN16', 'DN21', 'DN63',
'Double', 'Feather', 'Fish', 'Five', 'Fork', 'Gallatin', 'Grin', 'Haecksel',
'Hook', 'Jet', 'Jonah', 'Knit', 'Kringel', 'MN105', 'MN23', 'MN60', 'MN83',
'Mus', 'Notch', 'Number1', 'Oscar', 'Patchback', 'PL', 'Quasi', 'Ripplefluke',
'Scabs', 'Shmuddel', 'SMN5', 'SN100', 'SN4', 'SN63', 'SN89', 'SN9', 'SN90',
'SN96', 'Stripes', 'Thumper', 'Topless', 'TR120', 'TR77', 'TR82', 'TR88',
'TR99', 'Trigger', 'TSN103', 'TSN83', 'Upbang', 'Vau', 'Wave', 'Web',
'Whitetip', 'Zap', 'Zig', 'Zipfel'], ['Beak', 'Beescratch', 'Bumper', 'CCL',
'DN16', 'DN21', 'DN63', 'Double', 'Feather', 'Fish', 'Gallatin', 'Grin',
'Haecksel', 'Hook', 'Jet', 'Jonah', 'Knit', 'Kringel', 'MN105', 'MN60', 'MN83',
'Mus', 'Notch', 'Number1', 'Oscar', 'Patchback', 'PL', 'Ripplefluke', 'Scabs',
'Shmuddel', 'SN100', 'SN4', 'SN63', 'SN89', 'SN9', 'SN90', 'SN96', 'Stripes',
```

```
'Thumper', 'Topless', 'TR120', 'TR77', 'TR88', 'TR99', 'Trigger', 'TSN103',
'TSN83', 'Upbang', 'Vau', 'Wave', 'Web', 'Zap', 'Zipfel'], ['Beak',
'Beescratch', 'Bumper', 'CCL', 'DN16', 'DN21', 'DN63', 'Double', 'Feather',
'Fish', 'Gallatin', 'Grin', 'Haecksel', 'Hook', 'Jet', 'Jonah', 'Knit',
'Kringel', 'MN105', 'MN60', 'MN83', 'Mus', 'Notch', 'Number1', 'Oscar',
'Patchback', 'PL', 'Scabs', 'Shmuddel', 'SN100', 'SN4', 'SN63', 'SN9', 'SN90',
'SN96', 'Stripes', 'Thumper', 'Topless', 'TR77', 'TR99', 'Trigger', 'TSN103',
'Upbang', 'Web', 'Zap'], ['Beak', 'Beescratch', 'DN21', 'DN63', 'Double',
'Feather', 'Fish', 'Gallatin', 'Grin', 'Haecksel', 'Hook', 'Jet', 'Jonah',
'Knit', 'Kringel', 'MN105', 'MN83', 'Oscar', 'Patchback', 'PL', 'Scabs',
'SN100', 'SN4', 'SN63', 'SN9', 'SN90', 'SN96', 'Stripes', 'Topless', 'TR77',
'TR99', 'Trigger', 'TSN103', 'Upbang', 'Web', 'Zap']], [1.0, 0.8548387096774194,
0.7258064516129032, 0.5806451612903226])
```

14 Task 3b:

In our graph, we have a 'smelliness' attribute for each dolphin represented as an integer.

We'd like to conduct a study on the "smelliness" factor of dolphins and examine the correlation between the smelliness of dolphins and their ability to last in the community. Compute the average smelliness of dolphins that have survived each day, which will be returned in a list, where the first value is the average smelliness of all dolphins on day 0, the second is the average smelliness of all dolphins on day 1, and so on. Also, compute the correlation coefficient between a dolphin's number of days survived and the smelliness factor and return this as a float. The tutorial provides an example of how to accomplish this.

Does it appear that smelly dolphins last longer?

Disclaimer: The "smelliness" data and the survival mechanisms described in this question are entirely fictional and were created solely for educational purposes. Additionally, no dolphins were harmed in the creation of this exercise.

[42]:
```
task_id = '3b'

## Utility Code will go here
```

[54]:
```
def task_3b_solution():
    daily_survivors, _ = task_3a_solution()
    daily_avg_smelliness = [np.mean([G_dolphins.nodes[name]['smelliness'] for␣
 ↪name in G_dolphins.nodes()])]
    daily_avg_smelliness += [np.mean([G_dolphins.nodes[name]['smelliness'] for␣
 ↪name in daily_survivors[k]]) for k in range(len(daily_survivors))]
    correlation_coefficient = np.corrcoef(list(range(len(daily_survivors)+1)),␣
 ↪daily_avg_smelliness)[0,1]

    # YOUR CODE HERE
    #raise NotImplementedError()

    return daily_avg_smelliness, correlation_coefficient
```

[55]:
```
# Use this cell to explore your solution.

task_3b_solution()
```

[55]:
```
([31.612903225806452,
  31.612903225806452,
  36.15094339622642,
  40.15555555555556,
  44.94444444444444],
 0.9698163474786888)
```

[56]:
```
print(f"Task {task_id} - AG tests")

stu_ans = task_3b_solution()

print(f"Task {task_id} - your answer:\n{stu_ans}")

assert isinstance(stu_ans, tuple), f"Task {task_id}: Your function should␣
 ↪return a tuple."
assert len(stu_ans) == 2, f"Task {task_id}: Your returned tuple should have two␣
 ↪elements."

daily_avg_smelliness, correlation_coefficient = stu_ans

assert isinstance(
    daily_avg_smelliness, list
), f"Task {task_id}: The first element of the returned tuple should be a list."
assert all(
    isinstance(smell, float) for smell in daily_avg_smelliness
), f"Task {task_id}: Each element in `daily_avg_smelliness` should be a float."

assert isinstance(
    correlation_coefficient, float
), f"Task {task_id}: The second element of the returned tuple should be a float.
 ↪"

assert -1 <= correlation_coefficient <= 1, f"Task {task_id}: The correlation␣
 ↪coefficient should be between -1 and 1."
```

```
Task 3b - AG tests
Task 3b - your answer:
([31.612903225806452, 31.612903225806452, 36.15094339622642, 40.15555555555556,
44.94444444444444], 0.9698163474786888)
```

[ ]: